A MACHINE-CHECKED FINITE PERIODIC-LATTICE GAUSSIAN COVARIANCE

THE AIMATH PROJECT

Abstract. The free Gaussian field on a finite periodic lattice is the base layer of every constructive construction of $P(\varphi)_2$, and a formalization programme has to own that layer before it can own anything above it. We report a milestone: a self-contained, machine-checked development of the finite periodic-lattice Gaussian covariance in clean Lean 4 types. The covariance is not postulated from a spectral representation; it is built from the concrete operator $Q = -\Delta_a + m^2$ on $(\mathbb{Z}/N\mathbb{Z})^d$, whose self-adjointness, negative semidefiniteness, positive definiteness, Hermitian matrix form, and positive eigenvalues are all proved, and whose eigenbasis then yields the covariance $Q^{-1/2}$ with the Glimm–Jaffe normalization $a^{-d/2}$. The associated measure is the pushforward of finitely many independent standard Gaussians into the cylindrical weak dual, and its exact second moments, measurability, and L^2 membership are proved, including the exact finite formula in the mass eigenbasis and the second moment of the lifted configuration for an arbitrary family of continuous linear evaluation functionals. A change of representation identifies the factor with the upstream ℓ^2 coefficient formula, and a textual check against a pinned upstream commit certifies the provenance of the migrated definitions. The development comprises 34 modules and 149 theorem-level declarations over 3701 lines, with zero `sorry` and zero project axioms; 64 declarations report a footprint inside {`propext`, `Classical.choice`, `Quot.sound`}, the build is 8878 jobs, and 41 gate checks pass with parser negative controls. We state precisely what is *not* claimed: no continuum limit, no infinite volume, no interacting measure, no Osterwalder–Schrader reconstruction, and no identification with the upstream infinite-noise Gaussian implementation.

1. Why the free lattice layer is a milestone

1.1. **The object everyone else stands on.** A constructive construction of the two-dimensional interacting scalar field $P(\varphi)_2$ is a pipeline: start from the free Gaussian measure on a finite periodic lattice, insert a Wick-ordered polynomial interaction, prove that the normalized interacting measure has the Osterwalder–Schrader (OS) properties, and then take an infinite-volume limit and a continuum limit [GJ87, Sim74, GRS75]. Every later estimate in that pipeline consumes the *free* layer twice: once as the reference measure against which the interaction is defined, and once as the covariance whose decay and whose volume-uniform bounds make the limiting arguments work. Nelson's exponential bound, reflection positivity, the cluster expansion, and the tightness arguments all begin with a second-moment computation for the free field.

It follows that a formalization programme has an order of work whether it wants one or not. Before any interacting object can be stated, the following must exist as a machine-checked object: a lattice, an operator whose inverse is the covariance, an orthonormal eigenbasis with strictly positive eigenvalues, a Gaussian probability measure whose two-point function is that covariance, and a way to test the measure against continuum test functions. This paper reports the completion of exactly that layer, in clean types, for the finite periodic lattice in arbitrary dimension.

1.2. **Why the obvious source does not suffice.** The finite periodic-lattice construction is available in the upstream `GaussianField` development [Dou26], and our package migrates definitions

Date: 2026-10-21.

2020 *Mathematics Subject Classification.* 81T08, 60G15, 68V20, 65D30.

Key words and phrases. constructive quantum field theory, $P(\varphi)_2$, lattice Gaussian measure, Lean 4, formalization, spectral covariance.

from it at a pinned commit (`d63a28568a75d99f6cb27af1f888a49a69855a66`). Reusing it as-is would however have imported two things that the milestone cannot afford.

(i) *An axiom footprint and an infinite-noise representation.* The upstream covariance is reached through an infinite-dimensional Gaussian implementation. Whatever the mathematical merits of that route, a base layer that a downstream formalization is supposed to *stand on* should be re-derivable from finite data, with a footprint small enough to state in one line.

(ii) *A representation that does not match the consumers.* Upstream records the covariance in spectral coordinates: an ℓ^2 sequence on $\mathbb{N}$, obtained by zero-extending finite coefficient vectors. Our clean downstream interfaces instead take a continuous linear factor $T\colon (\Lambda \to \mathbb{R}) \to_{\mathcal{L}} H$ into a *finite* Euclidean space, push finitely many independent Gaussians through it, and ask for moments of the lifted configuration. Moving between these two descriptions is not a formality: it is an isometry between two inner-product spaces that has to be proved, not assumed.

The migration therefore has to do three separate things, and this paper is organized around the fact that they are separate: reconstruct the spectral data from the concrete operator (§3), build the measure and its moments in the consumer's representation (§5), and prove that the two representations agree (§6).

1.3. **What is new here, and what is not.** The mathematics of this paper is classical: Gaussian measures on finite-dimensional spaces, and the discrete Laplacian on a torus. Nothing in §3–§5 would surprise a reader of Glimm–Jaffe. What is new is the artifact and its certification:

C1 (concrete operator, §3):
The mass operator $Q = -\Delta_a + m^2$ is defined on functions on $(\mathbb{Z}/N\mathbb{Z})^d$ and comes with the full discrete-integration-by-parts package: self-adjointness, the directional sum-of-squares identity, negative semidefiniteness of Δ_a, and strict positive definiteness of Q.

C2 (spectral data from the operator, §3):
The orthonormal eigenbasis and the eigenvalues are *derived*, not assumed: a bridge lemma identifies the operator with its matrix in the delta basis, which is what lets the finite-dimensional spectral theorem apply; eigenvalues are then proved strictly positive. No compact-operator theory and no domain conditions enter.

C3 (covariance and normalization, §4):
The covariance is $Q^{-1/2}$ as a continuous linear map into a finite Euclidean space, with an exact bilinear identity, an explicit kernel, and the Glimm–Jaffe normalization by $a^{-d/2}$ proved to scale the bilinear form by a^{-d}.

C4 (measure and exact moments, §5):
The measure is a pushforward of finitely many independent standard Gaussians into the cylindrical weak dual. Its exact second moments, its measurability, and its L^2 membership are proved; the covariance identity is proved by expanding $(f + g)^2$ rather than by invoking an abstract polarization lemma, which keeps the load-bearing content where it belongs, in integrability.

C5 (agreement of representations, §6):
An explicit coefficient embedding into $\ell^2(\mathbb{N})$ is proved to be an isometry, and the migrated factor is proved to equal the upstream coefficient formula in that representation. The provenance of the migrated *definitions* is certified separately, by a textual comparison against the pinned upstream commit with corruption negative controls — a test, deliberately not a proof.

C6 (consumer interface, §7):
The lift from lattice configurations to configurations on an arbitrary test space is continuous and measurable, equals evaluation on the induced test field, and transports L^2 bounds, uniformly over a family of lattice spacings and volumes.

C7 (the audit, §9):
The claims above are not merely asserted to hold in Lean. A firewall checks that no project axiom and no `sorry` enters, with a negative control that injects an axiom and must be caught;

an interface gate re-queries every public declaration; a hash manifest pins the delivered bytes. The audit is the part of the package that makes the word "milestone" mean something.

1.4. **What this paper does not claim.** A formalization milestone is defined as much by its boundary as by its content. This package formalizes a *free*, *finite-volume*, *lattice* object. It asserts nothing about a continuum limit $a \to 0$, nothing about an infinite-volume limit $N \to \infty$, nothing about exponential clustering, nothing about an interacting measure, nothing about OS reconstruction, and nothing about the equality of the whole upstream infinite-noise Gaussian implementation with this finite pushforward law. Section 8 states each exclusion and what would be needed to remove it. Readers who take one sentence from this paper should take that one.

1.5. **Audience, conventions, and reading order.** The paper is written for two audiences at once: a mathematical physicist who wants to know exactly which statements are now machine-checked and with what footprint, and a formalization engineer who wants to know how the layer is built so it can be extended. Accordingly, each result appears in three registers: a mathematical statement with a proof (or a proof sketch that names the load-bearing step), the Lean identifier that carries it, and — in Appendix A — the module and kind of that identifier. Numbers in the prose are generated macros, recomputed from the delivered archive by `audit/count_lean.py`; Appendix B says how to re-run everything.

Conventions: $\Lambda = (\mathbb{Z}/N\mathbb{Z})^d$ with $N \geq 1$ and $d \geq 1$; $a > 0$ is the lattice spacing and $m > 0$ the mass; $\langle \cdot, \cdot \rangle$ is the Euclidean inner product of whichever real Hilbert space is at hand, always written with the space clear from context. Inner products on function spaces are *unweighted* sums over sites; the physical cell volume a^d is carried by the covariance normalization and, when it is wanted, by the sampling convention. Nothing below uses $d = 2$: the development is dimension-agnostic, and $d = 2$ is simply the instance a $P(\varphi)_2$ programme needs.

2. Setting and conventions

Everything in this paper lives on a finite object, and the finiteness is used everywhere: it replaces spectral theory of unbounded operators by spectral theory of Hermitian matrices, and it replaces Gaussian measures on nuclear spaces by finite-dimensional Gaussians pushed forward along a linear map. We therefore begin by fixing exactly which finite objects are in play.

2.1. **The lattice.**

Convention 2.1 (Sites and fields)**.** Let $d \geq 1$ and $N \geq 1$ be integers. The *lattice* is

$$\Lambda \;=\; (\mathbb{Z}/N\mathbb{Z})^d, \qquad |\Lambda| \;=\; N^d,$$

and a *configuration* is a real-valued function on Λ. We write $\mathcal{L}(\Lambda) = \Lambda \to \mathbb{R}$ for the configuration space and $e_i \in \Lambda$ for the unit vector in the i-th coordinate ($1 \leq i \leq d$). Addition in Λ is coordinatewise and modulo N.

In Lean the site type is `FinLatticeSites` and a field is `FinLatticeField`; the modulus hypothesis is a `NeZero` instance, so $N = 0$ is excluded by the type rather than by a side condition. The choice of $\mathbb{Z}/N\mathbb{Z}$ is not cosmetic: it makes *every* shift $x \mapsto x \pm e_i$ a bijection of Λ, which is the only property the summation-by-parts arguments of §3 use. A lattice with boundary would force those arguments to carry boundary terms, and the covariance would stop being translation invariant — a different, and strictly harder, milestone.

Convention 2.2 (Inner products)**.** For $f, g \colon \Lambda \to \mathbb{R}$ we write

$$\langle f, g \rangle_\Lambda \;=\; \sum_{x \in \Lambda} f(x)\, g(x), \qquad \|f\|_\Lambda^2 \;=\; \langle f, f \rangle_\Lambda .$$

The sum is *unweighted*: no cell volume enters the inner product. The physical volume a^d of a cell is carried by the covariance normalization of §4 and, where a Riemann-sum convention is wanted, by the sampling functional supplied to the lift (§7). This separation is deliberate; conflating the two is the most common way to lose a factor of a^d in lattice-to-continuum bookkeeping.

Convention 2.3 (Spacing, mass, and the operator)**.** Fix a lattice spacing $a > 0$ and a mass $m > 0$. The *discrete Laplacian* with spacing a acts by

$$(\Delta_a f)(x) \;=\; a^{-2} \sum_{i=1}^{d} \big[f(x + e_i) + f(x - e_i) - 2f(x) \big], \tag{1}$$

and the *mass operator* is

$$Q \;=\; -\Delta_a \;+\; m^2 \,\mathrm{id}. \tag{2}$$

Both are endomorphisms of $\mathcal{L}(\Lambda)$; both are continuous for the unique locally convex topology of the finite-dimensional space $\mathcal{L}(\Lambda)$, so no continuity hypothesis is ever needed. In Lean they are `finiteLaplacianFun` (the formula), `finiteLaplacian` (the same map as a continuous linear endomorphism), and `massOperator`.

Remark 2.4 (Finiteness is the enabling hypothesis)*.* Because Λ is finite, $\mathcal{L}(\Lambda)$ is a finite-dimensional real Hilbert space and Q is an ordinary Hermitian matrix in disguise. This is what allows §3 to obtain a complete orthonormal eigenbasis from Mathlib's spectral theorem for Hermitian matrices, without any of the domain theory that an unbounded operator on $L^2(\mathbb{R}^d)$ would require. The price is that the covariance is only ever defined at finite N; the absence of any continuum statement in this paper is a consequence of the method, not an oversight.

2.2. **Configurations as a weak dual.** Downstream, a "configuration" is not a function on Λ but a functional on test functions, because that is the form in which Gaussian measures can be pushed forward along a continuum limit. The clean interface keeps this general.

Definition 2.5 (Test space and cylindrical weak dual)**.** Let E be a real topological vector space, written additively, with the property that addition is continuous (in Lean: `AddCommGroup`, `Module` over $\mathbb{R}$, `TopologicalSpace`, `IsTopologicalAddGroup`). The *configuration space* is the continuous dual,

$$\mathcal{E}' \;=\; E \to_{\mathcal{L}} \mathbb{R},$$

read as a topological vector space for the weak-$*$ topology, and the *cylindrical σ-algebra* on it is the smallest σ-algebra making all evaluations $\omega \mapsto \omega(f)$ measurable, $f \in E$. For a family of continuous linear functionals $\mathrm{ev}_x \colon E \to \mathbb{R}$, $x \in \Lambda$, and $f \in E$ we write

$$\mathrm{ev}(f) \,:\, \Lambda \to \mathbb{R}, \qquad \mathrm{ev}(f)(x) \;=\; \mathrm{ev}_x(f),$$

and call it the *induced test field*. In Lean the space is `WeakDual` over $\mathbb{R}$, with the instance `inst MeasurableSpaceWeakDual`, and the induced test field is `latticeTestField`.

Remark 2.6 (How weak the hypotheses are)*.* Nothing in §7 uses local convexity, completeness, or separability of E. The reason is that only two operations on $\mathcal{E}'$ are ever performed: evaluation at a fixed test function, and pushing a measure forward along a map built from finitely many such evaluations. Both are measurable for the cylindrical σ-algebra by construction. This is worth stating because the corresponding classical statements are usually phrased for nuclear spaces (Minlos' theorem), and a reader may reasonably suspect that a nuclearity hypothesis is hiding somewhere. It is not: nuclearity earns its keep when one wants a Gaussian measure on $\mathcal{E}'$ from a covariance form alone, and we never do that — our measures always come from finite pushforwards or from explicit series.

2.3. **The reference Gaussian.**

Definition 2.7 (Product Gaussian)**.** For $n \geq 1$ and a variance $\sigma \geq 0$, the *product Gaussian* on $\mathbb{R}^n$ is

$$\mu_{n,\sigma} \;=\; \bigotimes_{i=1}^{n} \mathcal{N}(0,\sigma),$$

the n-fold product of the centred one-dimensional Gaussian of variance σ. In Lean it is `productGaussian`; the fact that it is a probability measure is the instance `productGaussian_isProbability`, and the coordinate second moment $\int x_i^2 \,\mathrm{d}\mu_{n,\sigma} = \sigma$ is `productGaussian_coordinate_second_moment_eq`.

Throughout, a variance parameter is a variance, not a standard deviation: $\mathcal{N}(0,\sigma)$ has second moment σ. With $\sigma = 1$ we write $\mu_n = \mu_{n,1}$, which is the law of n independent standard Gaussians — the raw material of every measure in this paper.

2.4. **Notation.**

Symbol	Meaning
Λ, $\lvert\Lambda\rvert = N^d$	sites $(\mathbb{Z}/N\mathbb{Z})^d$, Convention 2.1
$\mathcal{L}(\Lambda)$, $\langle\cdot,\cdot\rangle_\Lambda$	fields and unweighted inner product
Δ_a, $Q = -\Delta_a + m^2$	discrete Laplacian, mass operator
λ_k, e_k	eigenvalues and orthonormal eigenvectors of Q
$T = Q^{-1/2}$, $T_{\mathrm{GJ}} = a^{-d/2}T$	spectral covariance, Glimm–Jaffe normalization
$\mathcal{E}'$, ω	test space dual (configurations), a configuration
ev_x, $\mathrm{ev}(f)$	evaluation functionals, induced test field
$\mu_{n,\sigma}$, μ_T	product Gaussian, pushed-forward Gaussian
$C(x,y)$	covariance kernel $= \sum_k \lambda_k^{-1} e_k(x) e_k(y)$

3. The mass operator on the finite torus

This section proves the three facts about Q that the covariance needs: that it is self-adjoint, that it is strictly positive, and that it has a complete orthonormal eigenbasis with positive eigenvalues. The proofs are elementary — summation by parts on a finite group, and the spectral theorem for Hermitian matrices — but two of them carry a design decision that is worth making explicit, because the rest of the paper depends on it: the eigenbasis is obtained for the *matrix* of Q, and a bridge lemma transfers the eigen-relations back to the operator.

3.1. **Summation by parts on a finite group.**

Lemma 3.1 (Shift invariance)**.** *For $f, g\colon \Lambda \to \mathbb{R}$ and $v \in \Lambda$,*

$$\sum_{x\in\Lambda} f(x)\, g(x+v) \;=\; \sum_{x\in\Lambda} f(x-v)\, g(x).$$

Proof. The map $x \mapsto x - v$ is a bijection of Λ (this is where the torus is used), so the left-hand side equals $\sum_x f(x-v)\, g((x-v)+v) = \sum_x f(x-v)\, g(x)$. In Lean this is the reindexing step inside `finiteLaplacian_selfAdjoint`. □

Proposition 3.2 (Self-adjointness)**.** *For all $f, g\colon \Lambda \to \mathbb{R}$,*

$$\langle f, \Delta_a g\rangle_\Lambda \;=\; \langle \Delta_a f, g\rangle_\Lambda .$$

Proof. By (1) it suffices to prove the identity for a single direction i and then sum over i. Write $e = e_i$. Expanding the definition of the direction-i Laplacian,

$$\sum_x f(x)\big[g(x+e) + g(x-e) - 2g(x)\big] = \sum_x f(x)g(x+e) + \sum_x f(x)g(x-e) - 2\sum_x f(x)g(x).$$

Applying Lemma 3.1 with $v = -e$ to the middle term turns it into $\sum_x f(x+e)g(x)$. The resulting expression is symmetric under exchanging f and g, so it equals the same expression computed with the roles reversed, which is the direction-i right-hand side. Summing over i and multiplying by a^{-2} proves the claim. In Lean: `finiteLaplacian_selfAdjoint`. □

Proposition 3.3 (Directional sum of squares)**.** *For $f\colon \Lambda \to \mathbb{R}$, a direction i, and $e = e_i$,*

$$\sum_{x\in\Lambda} f(x)\big[f(x+e) + f(x-e) - 2f(x)\big] \;=\; -\sum_{x\in\Lambda}\big[f(x+e) - f(x)\big]^2.$$

Proof. Expanding the left-hand side gives $\sum_x f(x)f(x+e) + \sum_x f(x)f(x-e) - 2\sum_x f(x)^2$. By Lemma 3.1 the middle term equals $\sum_x f(x+e)f(x)$, and by the same reindexing $\sum_x f(x+e)^2 = \sum_x f(x)^2$. Hence the left-hand side equals

$$2\sum_x f(x)f(x+e) - \sum_x f(x+e)^2 - \sum_x f(x)^2 \;=\; -\sum_x\big[f(x+e) - f(x)\big]^2. \qquad \square$$

Corollary 3.4 (The Laplacian is negative semidefinite)**.** *Let $a > 0$. Then for every $f\colon \Lambda \to \mathbb{R}$,*

$$\langle f, \Delta_a f\rangle_\Lambda \;=\; -a^{-2}\sum_{i=1}^{d}\sum_{x\in\Lambda}\big[f(x+e_i) - f(x)\big]^2 \;\le\; 0.$$

Proof. Sum Proposition 3.3 over the d directions and multiply by $a^{-2} > 0$. In Lean: `finite Laplacian_neg_semidefinite`. □

Corollary 3.4 is the lattice statement of $-\Delta \ge 0$: the operator $-\Delta_a$ is the quadratic form of the Dirichlet energy, and its kernel on $\mathcal{L}(\Lambda)$ consists exactly of the constants (for $a > 0$ and $d \ge 1$; this characterisation is not needed below and is not formalized).

Proposition 3.5 (Strict positivity of the mass operator)**.** *Let $a > 0$, $m > 0$. Then for every $f\colon \Lambda \to \mathbb{R}$,*

$$\langle f, Qf\rangle_\Lambda \;=\; -\langle f, \Delta_a f\rangle_\Lambda + m^2\,\|f\|_\Lambda^2 \;\ge\; m^2\,\|f\|_\Lambda^2,$$

with equality in the second inequality only if $f = 0$. In particular $\langle f, Qf\rangle_\Lambda > 0$ for $f \neq 0$.

Proof. The identity is bilinearity of the inner product. The bound is Corollary 3.4 together with $m^2 > 0$; if $f \neq 0$ then $\|f\|^2 > 0$ because a sum of squares vanishes only if every term does. In Lean: `massOperator_pos_def`. □

3.2. From the operator to its matrix, and back.

Definition 3.6 (Matrix of the mass operator)**.** Let $\delta_y\colon \Lambda \to \mathbb{R}$ be the Kronecker delta at y (in Lean `finLatticeDelta`). The *matrix* of Q is

$$Q(x,y) \;=\; \big(Q\,\delta_y\big)(x), \qquad x, y \in \Lambda,$$

a real $|\Lambda| \times |\Lambda|$ matrix; in Lean `massOperatorEntry` and `massOperatorMatrix`.

Lemma 3.7 (Operator equals matrix)**.** *For every $f\colon \Lambda \to \mathbb{R}$ and every $x \in \Lambda$,*

$$\big(Qf\big)(x) \;=\; \sum_{y\in\Lambda} Q(x,y)\,f(y).$$

Proof. Every field is a finite linear combination of deltas, $f = \sum_y f(y)\,\delta_y$, and Q is linear; evaluating at x and using Definition 3.6 gives the claim. In Lean: `massOperator_eq_matrix_mulVec`. □

Corollary 3.8 (The matrix is Hermitian)**.** *$Q(x,y) = Q(y,x)$ for all $x, y \in \Lambda$.*

Proof. Apply Proposition 3.2 to the pair (δ_y, δ_x): $\langle \delta_y, Q\delta_x\rangle = \langle Q\delta_y, \delta_x\rangle$, which reads $Q(x,y) = Q(y,x)$. In Lean: `massOperatorMatrix_isHermitian`. □

Theorem 3.9 (Spectral data from the concrete operator)**.** *Let $a > 0$, $m > 0$. There exist an orthonormal basis $(e_k)_{k\in\Lambda}$ of the Euclidean space $\mathcal{H}_\Lambda = \mathbb{R}^\Lambda$ and real numbers λ_k such that*

(i) *$Qe_k = \lambda_k e_k$ for every $k \in \Lambda$;*
(ii) *$\lambda_k > 0$ for every k;*
(iii) *(Parseval) for all $f, g\colon \Lambda \to \mathbb{R}$, $\sum_k \langle e_k, f\rangle_\Lambda \langle e_k, g\rangle_\Lambda = \langle f, g\rangle_\Lambda$;*
(iv) *(spectral form of the quadratic form) $\langle f, Qf\rangle_\Lambda = \sum_k \lambda_k \langle e_k, f\rangle_\Lambda^2$.*

Proof. By Corollary 3.8 the matrix Q is Hermitian over $\mathbb{R}$, so the finite-dimensional spectral theorem for Hermitian matrices supplies an orthonormal basis e_k of $\mathbb{R}^\Lambda$ consisting of eigenvectors, with real eigenvalues λ_k. Lemma 3.7 upgrades the matrix identity $Qe_k = \lambda_k e_k$ to the operator identity (i). For (ii): if $e_k \neq 0$ then $\lambda_k \|e_k\|^2 = \langle e_k, Qe_k\rangle > 0$ by Proposition 3.5. Item (iii) is the Parseval identity for an orthonormal basis, and (iv) follows from (i) and (iii) by bilinearity: $\langle f, Qf\rangle = \sum_k \langle e_k, f\rangle \langle e_k, Qf\rangle = \sum_k \lambda_k \langle e_k, f\rangle^2$. In Lean the objects are `massEigenvalues`, `massEigenvectorBasis`, and the identities are `massOperator_eigenCoeff_eq_eigenvalues_mul_eigenCoeff`, `massEigenbasis_sum_mul_sum_eq_site_inner`, `massOperator_quadratic_eq_spectral`, with positivity in `massOperatorMatrix_eigenvalues_pos`. The structure bundling the data for the covariance is `spectralData`. □

Remark 3.10 (Why the bridge lemma is not bookkeeping)*.* Without Lemma 3.7, Theorem 3.9(i) would be a statement about rows and columns, and the identities (iii)–(iv) — which are what the covariance actually uses — would have no operator to refer to. The lemma is the only place where the identity $f = \sum_y f(y)\delta_y$ is spent, and it is spent once, for the whole development. An alternative route would prove the spectral theorem for the self-adjoint operator Q directly; that route needs the finite-dimensional operator spectral theorem or the compact self-adjoint theory, and buys nothing here.

Remark 3.11 (What is not formalized about the spectrum)*.* For orientation: on the torus one classically has the two-sided bound $m^2 \le \lambda_k \le m^2 + 4da^{-2}$, the upper end being the top of the symbol $-4a^{-2}\sum_i \sin^2$. Only positivity is formalized here, and the paper uses nothing stronger. A quantitative lower bound $\lambda_k \ge m^2$ together with the resulting variance bound is added to the source tree after this archive was cut; it is outside the audited milestone and is mentioned again only in Remark 10.2.

4. The covariance as $Q^{-1/2}$, and its normalization

4.1. **Spectral covariance data.** The covariance is defined from the spectral data of Theorem 3.9, and from nothing else. This is what makes the layer "concrete": the data are not an input parameter that a caller may supply inconsistently, because in the intended instance they are *computed* from Q (Definition 4.1 below and Theorem 3.9). The general form is still worth isolating, because it is the form the proofs use and the form a different lattice (a different site type, a different operator) would reuse.

Definition 4.1 (Spectral covariance data)**.** A *spectral covariance datum* on a finite type ι consists of an orthonormal basis $(e_k)_{k\in\iota}$ of the finite Euclidean space $\mathbb{R}^\iota$ and a family of reals $(\lambda_k)_{k\in\iota}$ with $\lambda_k > 0$ for all k. In Lean: `SpectralCovarianceData`, a structure with fields `eigenbasis`, `eigenvalues`, `eigenvalues_pos`.

Definition 4.2 (The covariance factor)**.** Given a datum as above, the *covariance factor* T is the linear map

$$T\colon \mathbb{R}^\iota \longrightarrow \mathbb{R}^\iota, \qquad Tf \;=\; \sum_{k\in\iota} \lambda_k^{-1/2} \langle e_k, f\rangle \; e_k, \tag{3}$$

where $\langle\cdot,\cdot\rangle$ is the Euclidean inner product of $\mathbb{R}^\iota$. In Lean: `spectralCovariance`. When the datum is the one of Theorem 3.9 for the concrete Q, the factor is written `latticeCovariance`.

Equation (3) is the formula for $Q^{-1/2}$ in the eigenbasis. We emphasize what is and is not being invoked: no functional calculus is used, and no square root of an operator is constructed. The symbol "$Q^{-1/2}$" is a name for the map (3), and the two properties the rest of the paper needs — the bilinear identity and the scaling under the Glimm–Jaffe normalization — are proved directly from the orthonormality of e_k.

Proposition 4.3 (Exact bilinear identity)**.** *Let T be as in* (3)*. For all $f, g \in \mathbb{R}^\iota$,*

$$\langle Tf, Tg\rangle \;=\; \sum_{k\in\iota} \lambda_k^{-1}\,\langle e_k, f\rangle\,\langle e_k, g\rangle\,. \tag{4}$$

In the concrete lattice instance, the same identity is `latticeCovariance_inner` *(general case:* `spectralCovariance_inner`*).*

Proof. By (3), $Tf = \sum_k c_k(f)e_k$ with $c_k(f) = \lambda_k^{-1/2}\langle e_k, f\rangle$. Orthonormality of (e_k) gives $\langle Tf, Tg\rangle = \sum_k c_k(f)c_k(g)$, and $c_k(f)c_k(g) = \lambda_k^{-1}\langle e_k, f\rangle\langle e_k, g\rangle$ because $\lambda_k^{-1/2}\lambda_k^{-1/2} = \lambda_k^{-1}$ and $\lambda_k > 0$. □

Remark 4.4 (The kernel)*.* Define $C\colon \Lambda \times \Lambda \to \mathbb{R}$ by

$$C(x, y) \;=\; \sum_{k\in\Lambda} \lambda_k^{-1} e_k(x)\, e_k(y).$$

Expanding (4) in coordinates gives $\langle Tf, Tg\rangle = \sum_{x,y} f(x)\, C(x, y)\, g(y)$, so C is the integral kernel of the bilinear form. Mathematically C is the Green function of Q: $\sum_y C(x, y)(Qg)(y) = g(x)$, equivalently $T^2 = Q^{-1}$, which follows from Theorem 3.9 by applying both sides to e_k. This identification is used for reading the formulas; it is not a separate formalized statement, and nothing in the paper depends on it.

4.2. The Glimm–Jaffe normalization.

Definition 4.5 (Normalized factor)**.** With $a > 0$ and $d \geq 1$ as before, the *Glimm–Jaffe normalized* factor is

$$T_{\mathrm{GJ}} \;=\; \left(a^d\right)^{-1/2} T.$$

In Lean: `spectralCovarianceGJ` in general and `latticeCovarianceGJ` for the concrete instance.

Proposition 4.6 (Scaling of the bilinear form)**.** *For all $f, g \in \mathbb{R}^\iota$,*

$$\langle T_{\mathrm{GJ}} f, T_{\mathrm{GJ}} g\rangle \;=\; \left(a^d\right)^{-1}\,\langle Tf, Tg\rangle\,.$$

In Lean: `spectralCovarianceGJ_inner` *and* `latticeCovarianceGJ_inner`*.*

Proof. T_{GJ} is a scalar multiple of T by $(a^d)^{-1/2}$, so the bilinear form is multiplied by the square of that scalar, $(a^d)^{-1/2}\cdot(a^d)^{-1/2} = (a^d)^{-1}$, using $a > 0$ so that $a^d > 0$. □

Remark 4.7 (Why the convention is recorded as data)*.* A formalization cannot afford a convention that lives in the reader's head. The package therefore records the normalization as a structure field — the lattice spacing and the dimension are data, not constants (`LatticeCovarianceGJRescalingData`, §7) — and Proposition 4.6 is the theorem that says exactly what the convention does to the second moments. A consumer that wants a different normalization can compute its effect from the identity rather than guess it. Which normalization a *continuum limit* wants is a question about the limit, and this layer does not answer it (§8).

Remark 4.8 (Reading the concrete instance)*.* The concrete instance chains Definition 4.1 to Theorem 3.9: `spectralData` bundles the eigenbasis and eigenvalues of the mass matrix together with their positivity proof, and `latticeCovariance` is `spectralCovariance` applied to it. Thus the Lean term for the covariance contains the whole derivation of the eigenbasis — there is no hypothesis of the form "assume an orthonormal eigenbasis with positive eigenvalues exists". This is the formal counterpart of the claim in the introduction that the spectral data are derived rather than postulated.

5. The Gaussian measure and its exact moments

We now build the measure. The construction is short by design: finitely many independent standard Gaussians, pushed into the configuration space along the linear map that reads the covariance factor, and then tested against test functions. What takes work is the bookkeeping: measurability with respect to the cylindrical σ-algebra, L^2 membership, and — the point of the whole section — exactness of the two-point function, including the exact finite formula in the mass eigenbasis.

5.1. Configurations from finitely many Gaussians.

Definition 5.1 (Sampled configuration)**.** Let ι be a finite type, $n = |\iota|$, and let $T\colon \mathbb{R}^\iota \to \mathbb{R}^\iota$ be a linear map (a covariance factor, in the applications of §4). Fix a numbering $\sigma\colon \{1,\dots,n\} \to \iota$. For $z = (z_1,\dots,z_n) \in \mathbb{R}^n$ the *sampled configuration* ω_z is the continuous linear functional on $\mathbb{R}^\iota$ given by

$$\omega_z(f) \;=\; \sum_{k=1}^{n} z_k \,\big(Tf\big)_{\sigma(k)}, \qquad f \in \mathbb{R}^\iota, \tag{5}$$

where $(\cdot)_{\sigma(k)}$ denotes the $\sigma(k)$-coordinate. In Lean: `covarianceConfiguration`, with the evaluation formula `covarianceConfiguration_apply`.

Two features of (5) are worth naming. First, ω_z is built as a *strong* dual element and then regarded as a weak dual element; this is what makes measurability immediate, since every evaluation is a finite sum of coordinate projections. Second, the definition consults T only through the coordinates of Tf: two factors with the same image inner products produce the same law. This is the formal reason why the representation change of §6 can be expected to preserve the moments, and the theorems below confirm it.

Proposition 5.2 (Exact second moment of a linear functional)**.** *For $n \geq 1$, a variance $\sigma \geq 0$, and $c \in \mathbb{R}^n$,*

$$\int_{\mathbb{R}^n} \Big(\sum_{i=1}^{n} c_i z_i\Big)^2 \mathrm{d}\mu_{n,\sigma} \;=\; \sigma \sum_{i=1}^{n} c_i^2. \tag{6}$$

In Lean: `productGaussian_linear_second_moment_eq`.

Proof. The coordinates of $\mu_{n,\sigma}$ are independent, centred, and have variance σ. Expanding the square gives $\sum_{i,j} c_i c_j \int z_i z_j$; the diagonal terms contribute $c_i^2\sigma$, and the off-diagonal terms vanish because $\int z_i z_j = \int z_i \int z_j = 0$ for $i \neq j$ by independence and centredness. In Lean the same conclusion is reached through the variance: the $c_i z_i$ are independent and square-integrable, variances add, and the mean is zero, so the second moment equals the variance $\sum_i c_i^2\sigma$. □

Proposition 5.3 (A bound that does not use independence)**.** *With the same notation,*

$$\int_{\mathbb{R}^n} \Big(\sum_{i=1}^{n} c_i z_i\Big)^2 \mathrm{d}\mu_{n,\sigma} \;\leq\; n \Big(\sum_{i=1}^{n} c_i^2\Big)\sigma.$$

In Lean: `productGaussian_linear_second_moment_bound`.

Proof. Pointwise, Cauchy–Schwarz in $\mathbb{R}^n$ gives $(\sum_i c_i z_i)^2 \leq n \sum_i c_i^2 z_i^2$; integrating and using the coordinate second moments gives the claim. Only the coordinate second moments are used, not independence. □

Remark 5.4 (Why both are formalized)*.* The exact identity (6) is the load-bearing statement: every exact moment in this paper is an instance of it. The bound of Proposition 5.3 is kept because it documents what the cheapest argument yields, and because it survives in situations where independence is unavailable or expensive. Having both in the library makes the cost of

the exact statement visible: a factor of n, which is exactly the price of dropping the variance computation.

5.2. Measurability and L^2.

Proposition 5.5 (The sampled configuration is measurable)**.** *The map $z \mapsto \omega_z$ from $\mathbb{R}^n$ into the dual of $\mathbb{R}^\iota$ is measurable for the cylindrical σ-algebra.*

Proof. By definition of the cylindrical σ-algebra it suffices that $z \mapsto \omega_z(f)$ be measurable for each f. By (5) this map is a finite linear combination of the coordinate projections $z \mapsto z_k$, hence measurable. In Lean: `covarianceConfiguration_measurable`. □

Proposition 5.6 (L^2 membership)**.** *For every $f \in \mathbb{R}^\iota$, the function $z \mapsto \omega_z(f)$ belongs to $L^2(\mu_{n,1})$.*

Proof. It is a finite sum of products of a coordinate (which is L^2 for a Gaussian law) with a constant; L^2 is closed under finite sums. In Lean: `covarianceConfiguration_memLp`. □

Theorem 5.7 (Exact covariance of the sampled measure)**.** *For all $f, g \in \mathbb{R}^\iota$,*

$$\int_{\mathbb{R}^n} \omega_z(f)\,\omega_z(g)\,\mathrm{d}\mu_{n,1} \;=\; \langle Tf, Tg\rangle\,. \tag{7}$$

In Lean: `covarianceConfiguration_covariance`.

Proof. By Proposition 5.6 the three products $\omega(f)^2$, $\omega(g)^2$ and $\omega(f)\omega(g)$ are integrable. Since ω is linear in its argument pointwise, $\omega(f+g)^2 = \omega(f)^2 + \omega(g)^2 + 2\,\omega(f)\omega(g)$, so integrating and using Proposition 5.2 (equivalently, the computation of Theorem 5.7 for $f+g$, f and g) yields

$$\langle T(f+g), T(f+g)\rangle = \langle Tf, Tf\rangle + \langle Tg, Tg\rangle + 2\int \omega(f)\omega(g),$$

and the claim follows by expanding the left-hand side and using symmetry of the inner product. In Lean this is precisely the route taken: the square of $\omega(f+g)$ is expanded, the three exact second moments are substituted, and the resulting linear relation is closed by `linarith`. □

Remark 5.8 (Polarization, by hand)*.* Theorem 5.7 is the polarization identity for the bilinear form $(f,g) \mapsto \int \omega(f)\omega(g)$, but it is proved by expanding a square rather than by invoking a general polarization lemma. The reason is methodological: the only non-formal input of the argument is integrability of the three products, i.e. Proposition 5.6; everything else is algebra. A proof that reaches for an abstract polarization statement would hide that dependence, and the dependence is the interesting part — it is what fails if one tries to run the same argument with a measure that only has a first moment.

Corollary 5.9 (Exact lattice formula)**.** *Let Q be the mass operator of §3 on Λ, let T_{GJ} be its Glimm–Jaffe normalized covariance factor (Definition 4.5), and let ω_z be as in* (5)*. Then for all $f, g\colon \Lambda \to \mathbb{R}$,*

$$\int \omega_z(f)\,\omega_z(g)\,\mathrm{d}\mu_{n,1} \;=\; \big(a^d\big)^{-1} \sum_{k\in\Lambda} \lambda_k^{-1} \Big(\sum_{x\in\Lambda} e_k(x) f(x)\Big)\Big(\sum_{x\in\Lambda} e_k(x) g(x)\Big). \tag{8}$$

In Lean: `latticeCovarianceGJ_gaussian_covariance`.

Proof. Combine Theorem 5.7 with Proposition 4.3 and Proposition 4.6, and read the inner products as sums over Λ. □

Definition 5.10 (The finite Gaussian measure)**.** The *finite covariance Gaussian measure* on the dual of $\mathbb{R}^\iota$ is the pushforward

$$\mu_T \;=\; (\omega_\cdot)_{\#}\,\mu_{n,1},$$

i.e. the image of the product Gaussian under the measurable map $z \mapsto \omega_z$. In Lean: `finite CovarianceGaussianMeasure`.

Proposition 5.11 (It is a probability measure, with the expected covariance)**.** *μ_T is a probability measure, and for all f, g,*

$$\int \omega(f)\,\omega(g)\,\mathrm{d}\mu_T \;=\; \langle Tf, Tg\rangle\,,$$

the integral being over the dual of $\mathbb{R}^\iota$. In Lean: the instance `finiteCovarianceGaussianMeasure_probability` *and the theorem* `finiteCovarianceGaussianMeasure_covariance`*.*

Proof. A pushforward of a probability measure is a probability measure. For the moments, apply the change-of-variables formula for a measurable map to Theorem 5.7; measurability of the integrand is Proposition 5.5. □

5.3. **The lift to a general test space.** The measure lives on the dual of $\mathbb{R}^\Lambda$. To compare it with anything continuum-like, it has to be tested against a general test space E, which is what the next definition does: each site carries a continuous linear evaluation functional, and the configuration is transported by summing the site values against those functionals.

Definition 5.12 (Finite-site lift)**.** Let ι be a finite type with decidable equality, E a test space as in Definition 2.5, and $\mathrm{ev}_x\colon E \to \mathbb{R}$ continuous linear for $x \in \iota$. The *lift* of a configuration ω on $\mathbb{R}^\iota$ is

$$\big(\mathrm{Lift}_{\mathrm{ev}}\omega\big)(f) \;=\; \sum_{x\in\iota} \omega(\mathbf{1}_x)\,\mathrm{ev}_x(f), \qquad f \in E, \tag{9}$$

where $\mathbf{1}_x$ is the indicator of the site x. In Lean: `finiteSiteEmbedLift` for the general site type and `latticeEmbedLift` for the $\mathrm{Fin}\,n$ interface, with the evaluation formula `latticeEmbedLift_eval`.

Proposition 5.13 (The lift is evaluation on the induced test field)**.** *$\mathrm{Lift}_{\mathrm{ev}}\omega$ is a continuous linear functional on E, the map $\omega \mapsto \mathrm{Lift}_{\mathrm{ev}}\omega$ is continuous for the weak-$*$ topologies and measurable for the cylindrical σ-algebras, and for every $f \in E$,*

$$\big(\mathrm{Lift}_{\mathrm{ev}}\omega\big)(f) \;=\; \omega\big(\mathrm{ev}(f)\big),$$

where $\mathrm{ev}(f) \in \mathbb{R}^\iota$ is the induced test field of Definition 2.5.

Proof. Continuity in f is a finite sum of continuous linear maps. Continuity in ω follows because each $\omega \mapsto \omega(\mathbf{1}_x)$ is a weak-$*$ evaluation, and measurability is the same statement for the cylindrical σ-algebra. The identity is a change of summation: writing $\mathrm{ev}(f) = \sum_x \mathrm{ev}_x(f)\mathbf{1}_x$ and using linearity of ω turns the right-hand side into the left-hand side. In Lean: `latticeEmbedLift_continuous`, `latticeEmbedLift_measurable`, and `latticeEmbedLift_eval_eq`; the general-site measurability is `finiteSiteEmbedLift_measurable`. □

Theorem 5.14 (Exact second moment of the lifted configuration)**.** *In the situation of Corollary 5.9, for every $f \in E$,*

$$\int \big(\mathrm{Lift}_{\mathrm{ev}}\omega\big)(f)^2\,\mathrm{d}\mu_{n,1} \;=\; \big(a^d\big)^{-1} \sum_{k\in\Lambda} \lambda_k^{-1} \Big(\sum_{x\in\Lambda} e_k(x)\,\mathrm{ev}_x(f)\Big)^2. \tag{10}$$

In Lean: `latticeCovarianceGJ_lift_second_moment`*; the same statement for the pushforward measure μ_T is* `latticeCovarianceGJ_pushforward_second_moment`*.*

Proof. By Proposition 5.13 the integrand equals $\omega(\mathrm{ev}(f))^2$, so the claim is Corollary 5.9 with $f = g = \mathrm{ev}(f)$. For the pushforward form, apply Proposition 5.11 instead. □

Remark 5.15 (What the lift does *not* decide). The family ev_x is an arbitrary family of continuous linear functionals. A physical sampling convention, or a Riemann-sum weight, has to be *supplied* through these maps; the theorem does not silently insert a lattice-volume weight. This is a deliberate refusal: the choice of sampling is part of the continuum model, and a base layer that made it silently

would be harder, not easier, to build on. In the next section the arbitrary family is what lets a single theorem cover both a pure point sampling and a weighted Riemann sum.

5.4. **A numbered form for the original interface.** Downstream code indexes finite sets by $\mathrm{Fin}\, n$, not by an abstract site type. The package therefore provides an explicit transport along a numbering of the sites, and proves that the transport is an isometry on the Hilbert side.

Definition 5.16 (Reindexed factor and numbered covariance)**.** For a finite type ι and a numbering $e\colon \iota \to \mathrm{Fin}\, n$, the reindexed factor is

$$\big(\mathrm{reindex}_e T\big) f \;=\; \big((T(f \circ e)) \circ e^{-1}\big) \qquad \text{as an element of } \mathbb{R}^{\mathrm{Fin}\, n},$$

and the numbered covariance is $\mathrm{reindex}_e T_{\mathrm{GJ}}$ for the canonical numbering of Λ. In Lean: `reindexCovariance` and `numberedLatticeCovarianceGJ`.

Proposition 5.17 (Transport preserves the moments)**.** $\langle \mathrm{reindex}_e T\, f, \mathrm{reindex}_e T\, g\rangle = \langle T(f \circ e), T(g \circ e)\rangle$ *for all f, g. Consequently the exact second moment of Theorem 5.14 holds verbatim for the original clean lift* `latticeEmbedLift` *on* $\mathrm{Fin}\, n$*. In Lean:* `reindexCovariance_inner` *and* `numberedLatticeCovarianceGJ_lift_second_moment`*.*

Proof. The first identity is a reindexing of a finite sum. The second statement substitutes the identity into Theorem 5.14 and rewrites $\mathrm{ev}_x(f)$ as $\mathrm{ev}_{e(x)}(f)$. □

With Proposition 5.17 the finite-lattice Gaussian is available on exactly the interface the clean downstream layer already uses, which is the point of the whole migration.

6. Agreement with the upstream spectral representation

The migrated covariance lives in $\mathbb{R}^{\Lambda}$, a finite Euclidean space indexed by *sites*. The upstream construction records the same coefficients in $\ell^2(\mathbb{N})$, indexed by the natural numbers, obtained by zero-extending a finite coefficient vector. A migration that did not relate the two would be a reimplementation, not a migration. This section proves the relation: an explicit coefficient embedding is an isometry, and under it the clean factor becomes exactly the upstream spectral formula.

Definition 6.1 (Coefficient inclusion and embedding)**.** Let ι be a finite type, $n = |\iota|$, and $e\colon \iota \to \mathrm{Fin}\, n$ a numbering. The *coefficient inclusion* is

$$\iota_{\ell^2}\colon \mathbb{R}^{\iota} \to \ell^2(\mathbb{N}), \qquad v \;\mapsto\; \sum_{k \in \iota} v_k\, \mathbf{e}_{e(k)},$$

where $\mathbf{e}_j$ is the j-th unit vector of $\ell^2(\mathbb{N})$, i.e. `lp.single 2 j 1` (zero outside its single nonzero coordinate). For a spectral datum D on ι, the *spectral coefficient embedding* is

$$\mathrm{Emb}_D \;=\; \iota_{\ell^2} \circ \mathrm{repr}_D,$$

where repr_D sends a vector to its coordinates in the eigenbasis e_k. In Lean: `finiteCoefficientInclusion` and `spectralCoefficientEmbedding`.

Proposition 6.2 (The embedding is an isometry)**.** *For all $v, w \in \mathbb{R}^{\iota}$,*

$$\langle \iota_{\ell^2} v, \iota_{\ell^2} w\rangle_{\ell^2(\mathbb{N})} = \langle v, w\rangle\,, \qquad \langle \mathrm{Emb}_D v, \mathrm{Emb}_D w\rangle_{\ell^2(\mathbb{N})} = \langle v, w\rangle\,.$$

In Lean: `finiteCoefficientInclusion_inner` *and* `spectralCoefficientEmbedding_inner`*.*

Proof. The vectors $\mathbf{e}_{e(k)}$, $k \in \iota$, are orthonormal in $\ell^2(\mathbb{N})$ because e is injective, so the inner product of the two finite sums is $\sum_k v_k w_k$. For the second identity, the representation map of an orthonormal basis is itself an isometry. □

Proposition 6.3 (The covariance in spectral coordinates)**.** *For $f \in \mathbb{R}^\iota$ and $k \in \iota$,*

$$\mathrm{repr}_D\big(\mathrm{cov}_D f\big)_k \;=\; \lambda_k^{-1/2} \sum_{x \in \iota} e_k(x)\, f(x),$$

hence

$$\mathrm{Emb}_D\big(\mathrm{cov}_D f\big) \;=\; \sum_{k \in \iota} \Big(\lambda_k^{-1/2} \sum_{x \in \iota} e_k(x) f(x)\Big) \mathbf{e}_{e(k)}.$$

In Lean: `spectralCovariance_repr` *and* `spectralCoefficientEmbedding_covariance`*.*

Proof. Read (3) in the eigenbasis: the coefficient of e_k is $\lambda_k^{-1/2} \langle e_k, f\rangle$, and $\langle e_k, f\rangle = \sum_x e_k(x) f(x)$ in coordinates. Applying the embedding and using its definition gives the second display. □

Theorem 6.4 (The migrated factor equals the upstream coefficient formula)**.** *Let Q be the concrete mass operator on Λ with spectral data (e_k, λ_k), let T_{GJ} be its Glimm–Jaffe normalized factor, and let e be the canonical numbering of Λ. Then for every $f\colon \Lambda \to \mathbb{R}$,*

$$\mathrm{Emb}\big(T_{\mathrm{GJ}} f\big) \;=\; \big(a^d\big)^{-1/2} \sum_{k \in \Lambda} \Big(\lambda_k^{-1/2} \sum_{x \in \Lambda} e_k(x) f(x)\Big) \mathbf{e}_{e(k)}. \tag{11}$$

In Lean: `latticeCovarianceGJ_upstream_coefficients`*.*

Proof. Combine Proposition 6.3 with the definition of T_{GJ} as the scalar $(a^d)^{-1/2}$ times the unnormalized factor; the scalar commutes with the (linear) embedding. □

Remark 6.5 (What Theorem 6.4 does and does not establish)*.* It establishes that the clean object and the upstream object are the same *formula* once the basis change is made explicit, and Proposition 6.2 says the basis change preserves every inner product, so every moment computed in one representation is available in the other. It does not establish that the upstream *construction* — an infinite-noise Gaussian implementation — coincides with the finite pushforward law of Definition 5.10. That would be a statement about two measures on the same space, and it is neither proved nor needed here; the package's README states the same boundary. A reader who wants the strongest honest summary: the *factor* agrees, the *moments* agree, the *measures* are not identified.

Remark 6.6 (Provenance is certified textually, not by proof)*.* Agreement of formulas could be a coincidence of notation. The package therefore also carries a source-correspondence check: it pins the upstream repository at commit `d63a28568a75d99f6cb27af1f888a49a69855a66`, refuses to run if that checkout is dirty or has moved, extracts the relevant definition bodies, normalizes whitespace, and compares them with the migrated bodies. The check excludes comments, compares only the finite-lattice definitions (not proofs), and includes two corruption negative controls — a formula with the inverse power removed and a formula with the wrong ℓ^2 index — which it must reject. This is a provenance test, not a Lean proof: it says where the definitions came from, not that they are correct. Section 9 explains why both kinds of evidence are needed and what neither can do.

7. The consumer interface: transporting L^2 bounds

A base layer earns its name by what it lets consumers prove without reopening it. The consumers in this project need one thing from a lattice approximation: a bound on the L^2 norm of a coordinate functional after the lattice configuration has been embedded into the continuum test space, holding *uniformly* in the lattice spacing and the volume. This section states the interface that delivers exactly that, and then states precisely which certificate hypotheses it does and does not discharge.

7.1. **Transport of a source bound.**

Lemma 7.1 (Pushforward L^2 transfer)**.** *Let μ be a measure on Ω, $\eta\colon \Omega \to \mathcal{E}'$ measurable, $f \in E$, and $g\colon \Omega \to \mathbb{R}$ with $\eta(\omega)(f) = g(\omega)$ for μ-almost every ω. If $\|g\|_{L^2(\mu)} \le C$ then*

$$\big\|\omega \mapsto \omega(f)\big\|_{L^2(\eta_\# \mu)} \;\le\; C.$$

In Lean: `pushforward_eval_L2_norm_le`*.*

Proof. The map η is measure preserving onto its image measure, so the L^2 seminorm of a function composed with η equals the seminorm of the function under the pushforward. The composition is g almost everywhere by hypothesis, and the seminorm is unchanged under almost-everywhere equality. □

Theorem 7.2 (Concrete finite-lattice transfer)**.** *Let μ be a measure on Ω, $\mathrm{embed}\colon \Omega \to \mathcal{E}'$ a measurable map into the dual of $\mathbb{R}^n$, $\mathrm{ev}_x\colon E \to \mathbb{R}$ continuous linear, $f \in E$, and $g\colon \Omega \to \mathbb{R}$ with*

$$\sum_{x=1}^{n} \mathrm{embed}(\omega)(\mathbf{1}_x)\,\mathrm{ev}_x(f) = g(\omega) \quad \text{for } \mu\text{-almost every } \omega.$$

If $\|g\|_{L^2(\mu)} \le C$, then the pushforward of μ along $\mathrm{Lift}_{\mathrm{ev}} \circ \mathrm{embed}$ satisfies $\big\|\omega \mapsto \omega(f)\big\|_{L^2} \le C$. The same holds uniformly over a family $\{\mu_i\}$, $\{\mathrm{embed}_i\}$, $\{g_i\}$ with a single constant C. In Lean: `latticeEmbedLift_pushforward_eval_L2_norm_le` *and* `latticeEmbedLift_uniform_pushforward_eval_L2_norm_le`*.*

Proof. The hypothesis identifies g with the composition of the lift with the embedding, evaluated at f; apply Lemma 7.1 to that composition, using measurability of the lift (Proposition 5.13). The uniform statement applies the result fibrewise at the same constant. □

Definition 7.3 (Uniform certificate bundle)**.** A *uniform lattice-Gaussian datum* consists of a family of measures μ_i on a common space, a family of measurable lifts into $\mathcal{E}'$, a family of lattice evaluations, a single real constant B, and the assertions that (i) each lift is measurable, (ii) each lift agrees almost everywhere with its lattice evaluation, and (iii) each source L^2 norm is bounded by B. In Lean: the structure `LatticeGaussianUniformData`, consumed by `latticeCovarianceGJ_uniform_data_eval_L2_norm_le`.

Remark 7.4 (What the interface asks of the caller). The bundle is deliberately shallow: it asks for measurability, an almost-everywhere identification, and one source bound. It does *not* ask for the covariance factor at all, which is why it can be instantiated by the concrete lattice of §5 and, in principle, by a different discretization. The price of that generality is that the source bound is a hypothesis, not a conclusion: this section transfers bounds, it does not produce them.

7.2. **Two covariance certificates.** The package also ships two certificate shapes that keep the covariance as an explicit field rather than a hypothesis on moments.

Definition 7.5 (GJ-shaped certificate)**.** A *GJ-shaped covariance certificate* consists of a continuous linear factor $T\colon \mathbb{R}^n \to H$ into a real Hilbert space H, a coefficient sequence $(v_k)_{k\in\mathbb{N}}$ in H, and a summability proof $\sum_k \|v_k\| < \infty$. In Lean: `LatticeCovarianceGJData`. The bound it yields, for a Gaussian measure built from the series, is `latticeCovarianceGJData_gaussian_pushforward_eval_L2_norm_le`; its shape is

$$\big\|\omega \mapsto \omega(f)\big\|_{L^2} \;\le\; \|\mathrm{id}\|_{L^2(\mathcal{N}(0,1))}\,\big(\|T(\mathrm{ev}(f))\|\ \textstyle\sum_k \|v_k\|\big).$$

Definition 7.6 (Rescaling certificate)**.** A *rescaling certificate* adds the lattice spacing and dimension as data, $T_{\mathrm{GJ}} = (a^d)^{-1/2} T_{\mathrm{bare}}$. In Lean: the structure `LatticeCovarianceGJRescalingData`

with the field `LatticeCovarianceGJRescalingData.latticeCovarianceGJ`. Two facts are proved about it: the bilinear form scales exactly,

$$\langle T_{\mathrm{GJ}} f, T_{\mathrm{GJ}} g\rangle = \left(a^d\right)^{-1} \langle T_{\mathrm{bare}} f, T_{\mathrm{bare}} g\rangle ,$$

which is `covariance_eq_inv_smul_bare`, and the diagonal is nonnegative (`covariance_self_nonneg`). The associated Gaussian measure is an instance of probability measure, and the L^2 bound of Definition 7.5 holds for it (`latticeCovarianceGJRescalingData_gaussian_pushforward_eval_L2_norm_le`, uniformly in `latticeCovarianceGJRescalingData_uniform_gaussian_pushforward_eval_L2_norm_le`).

Remark 7.7 (The two routes are not identified). There are now two ways to put a Gaussian measure on $\mathcal{E}'$ in this package. The first is the finite pushforward of Definition 5.10, which is exact but lives at finite N. The second is a Gaussian *series* construction: for a coefficient sequence (v_k) with $\sum_k \|v_k\| < \infty$, set $\omega(f) = \sum_k \xi_k \langle v_k, Jf\rangle$ for ξ an infinite product of standard Gaussians, which is almost surely summable and has the explicit bound `gaussianWeakDualMeasure_eval_L2_norm_bound`; the almost-sure series identity is `gaussianSeriesConfiguration_eval_eq_tsum_ae` and the measure is `gaussianWeakDualMeasure`. The two constructions are *not* proved equal in this archive, and this paper does not claim they are. That would be a statement about two different measures on the same space; the finite route is what carries the exact second moments, and the series route is what carries an ℓ^1-controlled L^2 bound. Keeping both, and keeping the non-identification explicit, is the honest state of the migration.

Remark 7.8 (Why this is the right interface for a tightness argument). Uniform L^2 bounds on coordinate functionals are exactly the hypothesis under which Chebyshev-type tail estimates and compact containment for families of measures are proved; the archive contains such finite-dimensional tightness lemmas as separate modules, stated for an abstract family of measures and functionals. What the finite-lattice milestone supplies is the *source* of those bounds for one concrete family. It does not itself perform the tightness argument, and no compactness claim is made in this paper (§8).

8. What is not claimed

A formalization is only as trustworthy as the list of things it does not say. This section is that list, with, for each item, the concrete work that removing it would require. The list is not a disclaimer; it is the specification of the next milestone.

- **(N1) No continuum limit.** Nothing here says anything about $a \to 0$. The covariance is defined at fixed $a > 0$, and the spacing enters only through Δ_a and the normalization $a^{-d/2}$. Removing this exclusion needs: uniform-in-a estimates on moments of the field beyond the second moment, a Wick-ordering subtraction to remove the coinciding-point divergence, tightness on a space of distributions, and an identification of the limit. None of that is in this layer, and the layer is built so that it does not have to be.
- **(N2) No infinite-volume limit.** Nothing here says anything about $N \to \infty$. Every object is a finite sum over Λ. Removing this exclusion needs volume-uniform bounds on exponential moments of the interaction — the Nelson bound [Nel73] and its layer-cake form — which the archive does contain as separate finite-volume statements for an abstract V, but which are not connected to the covariance of §4 in this paper.
- **(N3) No clustering and no mass gap.** The mass m is a parameter of the operator; no statement is made about a spectral gap of a transfer matrix, about exponential decay of connected correlations, or about a physical mass. Removing this exclusion needs the transfer-matrix construction together with a genuine spectral-gap or cluster-expansion argument.

(N4) No interacting measure. The measure of §5 is Gaussian. There is no Wick-ordered interaction, no partition function, no interacting Schwinger function. The archive contains an independent formalization of the Wick monomial (as the Hermite polynomial identity and its first few cases), but it is not used to construct an interacting measure here.

(N5) No Osterwalder–Schrader reconstruction. Nothing here asserts that any measure satisfies reflection positivity or the OS axioms [OS73], and nothing reconstructs a Wightman theory. The archive does contain standalone finite-dimensional lemmas about positivity-improving matrices and about reflection positivity being preserved by weak limits; they are not wired to the measure of §5 and are not used in this paper. Removing this exclusion needs both the measure-level statement and the reconstruction theorem itself.

(N6) No identification of the two Gaussian routes. As stated in Remark 7.7: the finite pushforward measure and the Gaussian series measure on the weak dual are two different objects, and the archive does not prove them equal. Removing this exclusion needs an infinite-noise Gaussian implementation on that space and an equality of laws — which is exactly the upstream construction this milestone deliberately does not depend on.

(N7) No canonical sampling convention. The lift accepts an arbitrary family of continuous linear evaluation functionals, and a physical sampling or Riemann-sum weight must be supplied through them (Remark 5.15). The theorems are neutral with respect to that choice: they neither fix it nor silently insert a factor. Removing this exclusion would mean committing to a specific discretization of test functions, which belongs to the continuum model, not to the base layer.

One further exclusion is methodological rather than mathematical, and it is discussed where it belongs: no check in §9 can establish that a formal statement is the *intended* one. Machine checking transfers confidence from proofs to statements only if the statements are read; that is what Sections 3–7 are for.

9. The audit: what is checked, and what a check cannot do

The word "machine-checked" is doing a lot of work in this paper, and it is worth saying exactly how much. A proof assistant guarantees that a proof term inhabits a type. It does not guarantee that the type says what the author meant, that the development was built from the sources it claims, or that the declarations it was asked about are the ones that matter. The package therefore ships four gates, each aimed at one of those failure modes, each with a negative control. This section describes them and then states what they leave open.

9.1. **Gate 1: the build.** The root module imports the whole library and is built with `lake build AImathPPhi2Clean` [LU21, mC20]. The archived log records a successful build of 8878 jobs at `leanprover/lean4:v4.34.0-rc2`, with Mathlib pinned in the Lake manifest at commit `852d20a84581aeba8527bd01d0be361d8c0e71c0`. The build emits 42 warnings; they are deprecation notices and style linter messages, are recorded in the log rather than suppressed, and are not correctness assertions. A build is the weakest of the four gates: it says the proof terms typecheck, nothing more.

9.2. **Gate 2: the axiom firewall.** For a curated list of public declarations, the package prints the axiom footprint with `#print axioms` and checks three things:

(i) every queried declaration appears in the report (no silently missing entries);
(ii) its axiom set is contained in the allowlist {`propext`, `Classical.choice`, `Quot.sound`} of size 3;
(iii) no other axiom appears.

Two details make this more than a formality.

The report is transitive. `#print axioms` walks the proof term, so a declaration whose proof uses a lemma that uses an axiom reports the axiom. In particular, `sorry` elaborates to an axiom named `sorryAx`; its absence from every reported set is therefore evidence of absence of `sorry` on every path to the queried declarations, not merely absence at the surface. Consistently, a comment-stripped scan of the delivered sources finds zero occurrences of the token `sorry` and 0 occurrences of an `axiom` declaration.

The detector is tested. A file `audit/negative_axiom.lean` declares a custom axiom, uses it, and prints its footprint; the gate passes only if the log reports `injectedAxiom`. A firewall that has never fired is indistinguishable from one that cannot fire.

The gates together query 64 declarations: 62 report the allowlisted set and 2 report no axioms at all (they are type abbreviations, which have no proof content). The per-module interface files cover 27 modules.

9.3. **Gate 3: the interface gate, and its own negative controls.** The interface gate re-runs every `audit/*\_interface.lean` file, each of which ends with `#print axioms` lines for the public declarations of one module. The gate re-derives the expected declaration list from the file text and rejects a report that is missing a declaration, that contains an unexpected one, that contains a forbidden axiom or `sorryAx`, that is empty, or that carries a Lean error anywhere.

The gate's *parser* is itself tested against four malformed reports: a report with an injected custom axiom, one with `sorryAx`, the empty string, and an error followed by a clean report. Each must be rejected, and the check fails if any is accepted. This is the pattern the whole audit follows: the check is exercised on an input it must refuse before it is trusted on an input it must accept. In total the deliverable log records 41 passing checks.

9.4. **Gate 4: source provenance and delivery integrity.** The migrated definitions are not merely similar to the upstream ones; the archive pins the upstream repository at commit `d63a2856 8a75d99f6cb27af1f888a49a69855a66`, refuses to proceed if that checkout has moved or if the four relevant files are dirty, strips comments and whitespace, and compares the definition *bodies* (not the proofs) with the migrated ones. Two deliberate corruptions — the inverse power dropped, and the ℓ^2 index changed — must be rejected by the same comparison. This is a provenance test: it does not prove anything, and Section 6 says so explicitly.

Finally, the archive carries a `SHA256SUMS` manifest for 117 files, and 117 of those entries match the bytes inside the archive. The delivery is pinned twice: by that manifest and by the SHA-256 of the archive itself. The registry entry that names the archive records the same hash, so a reader who has the archive can check that it is the object this paper describes.

9.5. **What none of the four gates establishes.**

(i) *Statement adequacy.* A theorem can be true and vacuous, or true and weaker than intended. No gate checks that a formal statement is the intended mathematical statement. The mitigation is human: the statements of Sections 3–7 are written out, and Appendix A maps each one to its identifier so that a reader can open the source and compare.

(ii) *Coverage.* The axiom gate queries a curated list. A declaration outside the list is covered only if some queried declaration depends on it. The interface files are organised by module to make the coverage broad, but breadth is a design goal, not a theorem.

(iii) *Adequacy of the allowlist.* That {`propext`, `Classical.choice`, `Quot.sound`} is an acceptable footprint is a policy judgement: these are the standard axioms of Lean and Mathlib, not project-specific mathematical assumptions. A reader who rejects classical choice will read the footprint differently — and can, because it is printed.

(iv) *Mathematical truth in a Platonic sense.* What is established is that the statements follow from the axioms of the ambient logic, not that the ambient logic is consistent, and not that the intended mathematical objects exist independently.

The fourth point is not a defect of this package; it is the permanent residue of every formalization. What can be reduced is the first: the statement challenge. The development mitigates it in three ways: statements carry explicit hypotheses and quantifiers (no implicit "for the usual lattice"); the paper supplies an independent human proof for each result, so a reader can check the mathematics without reading Lean; and the ledger is exhaustive in the sense that every Lean name cited in the body appears in it.

9.6. **Reproducibility, honestly.** The archived logs were produced on the project's Linux build machine with the toolchain pinned at `leanprover/lean4:v4.34.0-rc2`. Re-running the checks requires that toolchain, the Mathlib revision recorded in the manifest, an upstream `GaussianField` checkout at `d63a28568a75d99f6cb27af1f888a49a69855a66`, and two edits: the audit shell scripts name the Lean toolchain and a Python interpreter by absolute path, because the build machine needs them. Those two path variables are the only machine-specific content in the recipe, and they are the first thing to change when reproducing elsewhere. The archive also contains the full source, so a reader can build it as a standalone Lake project against the pinned Mathlib without the project's own scripts. A public mirror of the package — the same bytes, with the three machine-specific audit scripts made portable and the delta declared file by file — is available at https://github.com/springoxn/pphi2-clean-finite-lattice.

The numbers in this paper are not transcribed from the logs: they are computed from the archive's bytes by `audit/count_lean.py`, emitted as the macros used above, and re-checked by `audit/verify_paper.py`, which also fails if a derived count is typed into the prose by hand. The prose therefore never types a count itself: the numbers are macros, and a stale number is a build failure rather than a typo.

10. The milestone

We can now say what the milestone is. It is not a theorem; it is the availability, with the properties listed below, of a layer.

Criterion 10.1 (Finite-lattice Gaussian milestone)**.** The finite periodic-lattice free Gaussian layer is *available* for a $P(\varphi)_2$ formalization when there is a self-contained development in clean types that provides:

- **M1** a covariance derived from a concrete lattice operator, with no hypothesis postulating spectral data;
- **M2** a machine-checked footprint containing no project axiom and no `sorry`, certified by a check that has been shown to fire;
- **M3** exact second moments, in closed form in the mass eigenbasis, for the measure and for its lift;
- **M4** the layer on the interfaces the downstream consumers already use, including the numbered finite-set interface;
- **M5** a proved relation to the upstream spectral representation, with the provenance of the migrated definitions certified separately;
- **M6** a reproducible build and delivery, pinned by content hashes.

Table 1 records the status of each criterion and the evidence that decides it. The evidence column is deliberately mechanical: each row points at a declaration, a log, or a manifest, not at a paragraph of this paper.

TABLE 1. Acceptance record for Criterion 10.1. Every row points at a declaration, a log, or a manifest rather than at a paragraph of this paper; the caveats are the ones discussed in Sections 8 and 9.

	Criterion	Deciding evidence
M1	covariance from the concrete operator; spectral data derived, not assumed	`massOperator_eq_matrix_mulVec`, `spectralData`, Theorem 3.9
M2	no project axiom, no `sorry`; detector shown to fire	axiom report for 64 declarations over an allowlist of size 3, with the injected-axiom negative control
M3	exact moments, closed form in the eigenbasis	`covarianceConfiguration_covariance`, `latticeCovarianceGJ_gaussian_covariance`, `latticeCovarianceGJ_lift_second_moment`
M4	clean interfaces, including the numbered finite-set interface	`latticeEmbedLift`, `numberedLatticeCovarianceGJ`, and the lift moment for the numbered factor (Table 3)
M5	proved relation to the upstream representation	`spectralCoefficientEmbedding_inner`, `latticeCovarianceGJ_upstream_coefficients`; provenance test at commit `d63a28568a75d99f6cb27af1f888a49a69855a66` with two corruption controls
M6	reproducible build and pinned delivery	build of 8878 jobs; 41 gate checks; 117/117 manifest entries; the archive hash is recorded in Appendix C

10.1. **What changes, and what does not.** *What changes.* A downstream development can now write down the free lattice Gaussian as a term, obtain its two-point function as a theorem in the mass eigenbasis, lift it to an arbitrary test space, and transport L^2 bounds. In particular, the Gaussian layer of a $P(\varphi)_2$ construction no longer has to be assumed: it is constructible, and the construction is audited. For the project's proof graph this is the first time the free finite-lattice layer is *machine-checked in clean types* rather than merely proved on paper.

What does not change. The *mathematical* status of the $P(\varphi)_2$ programme. The statements formalized here were already known and already proved on paper; a formalization milestone moves the trusted base, not the frontier. It also does not raise the status of any interacting statement: those still need the continuum and infinite-volume layers, together with OS reconstruction (§8).

Why it is still a milestone. Because the criterion is availability, not novelty. A layer that is available can be consumed without re-derivation, and the whole point of the ordering argument in §1 is that everything else in the programme is downstream of this layer. The milestone is the moment the base of the tower stops being a hypothesis.

10.2. **Residual work, in the order it must be done.**

(1) *Wire the existing standalone modules to this covariance.* The archive already contains finite-volume Nelson layer-cake statements, Wick monomial identities, tightness lemmas for abstract families, and positivity-improving matrix lemmas. None of them is connected to the measure of §5 in this paper. Connecting them is mechanical but not trivial, and it is the cheapest next step.

(2) *Quantitative lattice bounds.* Uniform-in-N and uniform-in-a bounds on low moments of the covariance are the quantitative input of any tightness argument. This is the natural next formalization unit.
(3) *The interacting finite-volume measure.* Wick ordering, the Boltzmann weight, and the normalized measure, together with its first moments.
(4) *The infinite-volume and continuum limits, and OS reconstruction.* These are the hard analytic layers, and they are where the mathematical risk of the programme actually lives.

Remark 10.2 (The state of the live source tree). This paper describes the delivered archive, because the archive is the object that can be pinned and audited. The live source tree continues to move: at the time of writing it differs from the archive in 2 files and carries 3 additional module(s), one of which proves a quantitative lower bound on the eigenvalues and the corresponding bound on the variance of a physically sampled test field. Those modules are outside the audited milestone, their statements are not claimed here, and none of the numbers in this paper include them. The generation script reports the drift precisely so that a reader is never left guessing which snapshot a number came from.

Remark 10.3 (How to read this paper against the sources). Every result in Sections 3–7 carries its Lean identifier. Appendix A lists those identifiers with their module and kind; Appendix B lists the checks, what they establish, and what they cannot. A reader who wants to verify a single claim should open the module, find the declaration, and check that the statement printed here and the statement there agree — that comparison is precisely the step no gate in this package performs, and therefore the step that a human reader is best placed to do.

Appendix A. Declaration ledger

Every Lean name cited in the body of this paper appears in Table 2, with the kind of declaration it is and the module that declares it. The table is not maintained by hand: it is generated from the body's citations and resolved against the declaration index of the delivered archive, and the generator fails the build if a cited name does not resolve. A reader who wants to check a single claim can therefore go from the paper's statement to the source in one step.

Two caveats about what the table is. First, it is a *name* ledger: it proves that the declaration exists and where it lives, not that its statement matches the prose. That comparison is the one step no script in this package performs (§9), and it is why the statements are printed in full above. Second, the kinds are read off the source syntax; a `theorem` and a `lemma` are both proved declarations, and the distinction between them in Lean is stylistic.

Table 3, at the end of this appendix, gives the coarser map: from the numbered results of the paper to their identifiers, for readers who want the principal statements without the noise.

Table 2: Declaration ledger: every Lean name cited in the body, with its kind and module.

Declaration (as cited)	Kind	Module
Setting and conventions		
`FinLatticeSites`	abbrev	`ConcreteLatticeOperator`
`FinLatticeField`	abbrev	`ConcreteLatticeOperator`
`finiteLaplacianFun`	def	`ConcreteLatticeOperator`
`finiteLaplacian`	def	`ConcreteLatticeOperator`
`massOperator`	def	`ConcreteLatticeOperator`
`instMeasurableSpaceWeakDual`	instance	`GaussianConfiguration`
`latticeTestField`	def	`LatticeGaussianContinuumConsumer`

Declaration (as cited)	Kind	Module
`productGaussian`	def	`ProductGaussian`
`productGaussian_isProbability`	instance	`ProductGaussian`
`productGaussian_coordinate_second_moment_eq`	theorem	`ProductGaussian`
The mass operator		
`finiteLaplacian_selfAdjoint`	theorem	`ConcreteLatticeOperator`
`finiteLaplacian_neg_semidefinite`	theorem	`ConcreteLatticeOperator`
`massOperator_pos_def`	theorem	`ConcreteLatticeOperator`
`finLatticeDelta`	def	`ConcreteLatticeOperator`
`massOperatorEntry`	def	`ConcreteLatticeOperator`
`massOperatorMatrix`	def	`ConcreteLatticeOperator`
`massOperator_eq_matrix_mulVec`	theorem	`ConcreteLatticeOperator`
`massOperatorMatrix_isHermitian`	theorem	`ConcreteLatticeOperator`
`massEigenvalues`	def	`ConcreteLatticeOperator`
`massEigenvectorBasis`	def	`ConcreteLatticeOperator`
`massOperator_eigenCoeff_eq_eigenvalues_mul_eigenCoeff`	theorem	`ConcreteLatticeOperator`
`massEigenbasis_sum_mul_sum_eq_site_inner`	theorem	`ConcreteLatticeOperator`
`massOperator_quadratic_eq_spectral`	theorem	`ConcreteLatticeOperator`
`massOperatorMatrix_eigenvalues_pos`	theorem	`ConcreteLatticeOperator`
`spectralData`	def	`ConcreteLatticeOperator`
The covariance and its normalization		
`SpectralCovarianceData`	structure	`RealLatticeSpectralCovariance`
`spectralCovariance`	def	`RealLatticeSpectralCovariance`
`latticeCovariance`	def	`ConcreteLatticeCovariance`
`latticeCovariance_inner`	theorem	`ConcreteLatticeCovariance`
`spectralCovariance_inner`	theorem	`RealLatticeSpectralCovariance`
`spectralCovarianceGJ`	def	`RealLatticeSpectralCovariance`
`latticeCovarianceGJ`	def	`ConcreteLatticeCovariance`
`spectralCovarianceGJ_inner`	theorem	`RealLatticeSpectralCovariance`
`latticeCovarianceGJ_inner`	theorem	`ConcreteLatticeCovariance`
`LatticeCovarianceGJRescalingData`	structure	`LatticeGaussianContinuumConsumer`
`spectralData`	def	`ConcreteLatticeOperator`
The Gaussian measure and its moments		
`covarianceConfiguration`	def	`ConcreteGaussianMoment`
`covarianceConfiguration_apply`	theorem	`ConcreteGaussianMoment`
`productGaussian_linear_second_moment_eq`	theorem	`ProductGaussianLinearMoment`
`productGaussian_linear_second_moment_bound`	theorem	`ProductGaussianLinearMoment`
`covarianceConfiguration_measurable`	theorem	`ConcreteGaussianMoment`
`covarianceConfiguration_memLp`	theorem	`ConcreteGaussianMoment`
`covarianceConfiguration_covariance`	theorem	`ConcreteGaussianMoment`
`latticeCovarianceGJ_gaussian_covariance`	theorem	`ConcreteGaussianMoment`
`finiteCovarianceGaussianMeasure`	def	`ConcreteGaussianMoment`
`finiteCovarianceGaussianMeasure_probability`	instance	`ConcreteGaussianMoment`
`finiteCovarianceGaussianMeasure_covariance`	theorem	`ConcreteGaussianMoment`
`finiteSiteEmbedLift`	def	`RealLatticeSpectralCovariance`
`latticeEmbedLift`	def	`LatticeGaussianContinuumConsumer`
`latticeEmbedLift_eval`	theorem	`LatticeGaussianContinuumConsumer`

Declaration (as cited)	Kind	Module
latticeEmbedLift_continuous	theorem	LatticeGaussianContinuumConsumer
latticeEmbedLift_measurable	theorem	LatticeGaussianContinuumConsumer
latticeEmbedLift_eval_eq	theorem	LatticeGaussianContinuumConsumer
finiteSiteEmbedLift_measurable	theorem	ConcreteGaussianMoment
latticeCovarianceGJ_lift_second_moment	theorem	ConcreteGaussianMoment
latticeCovarianceGJ_pushforward_second_moment	theorem	ConcreteGaussianMoment
reindexCovariance	def	NumberedLatticeGaussianMoment
numberedLatticeCovarianceGJ	def	NumberedLatticeGaussianMoment
reindexCovariance_inner	theorem	NumberedLatticeGaussianMoment
numberedLatticeCovarianceGJ_lift_second_moment	theorem	NumberedLatticeGaussianMoment
Agreement of representations		
finiteCoefficientInclusion	def	LatticeCovarianceRepresentation
spectralCoefficientEmbedding	def	LatticeCovarianceRepresentation
finiteCoefficientInclusion_inner	theorem	LatticeCovarianceRepresentation
spectralCoefficientEmbedding_inner	theorem	LatticeCovarianceRepresentation
spectralCovariance_repr	theorem	LatticeCovarianceRepresentation
spectralCoefficientEmbedding_covariance	theorem	LatticeCovarianceRepresentation
latticeCovarianceGJ_upstream_coefficients	theorem	LatticeCovarianceRepresentation
The consumer interface		
pushforward_eval_L2_norm_le	theorem	LatticeGaussianContinuumConsumer
latticeEmbedLift_pushforward_eval_L2_norm_le	theorem	LatticeGaussianContinuumConsumer
latticeEmbedLift_uniform_pushforward_eval_L2_norm_le	theorem	LatticeGaussianContinuumConsumer
LatticeGaussianUniformData	structure	LatticeGaussianContinuumConsumer
latticeCovarianceGJ_uniform_data_eval_L2_norm_le	theorem	LatticeGaussianContinuumConsumer
LatticeCovarianceGJData	structure	LatticeGaussianContinuumConsumer
latticeCovarianceGJData_gaussian_pushforward_eval_L2_norm_le	theorem	LatticeGaussianContinuumConsumer
LatticeCovarianceGJRescalingData	structure	LatticeGaussianContinuumConsumer
LatticeCovarianceGJRescalingData.latticeCovarianceGJ	def	LatticeGaussianContinuumConsumer
covariance_eq_inv_smul_bare	theorem	LatticeGaussianContinuumConsumer
covariance_self_nonneg	theorem	LatticeGaussianContinuumConsumer
latticeCovarianceGJRescalingData_gaussian_pushforward_eval_L2_norm_le	theorem	LatticeGaussianContinuumConsumer
latticeCovarianceGJRescalingData_uniform_gaussian_pushforward_eval_L2_norm_le	theorem	LatticeGaussianContinuumConsumer
gaussianWeakDualMeasure_eval_L2_norm_bound	theorem	GaussianConfiguration
gaussianSeriesConfiguration_eval_eq_tsum_ae	theorem	GaussianConfiguration
gaussianWeakDualMeasure	def	GaussianConfiguration
The milestone		
massOperator_eq_matrix_mulVec	theorem	ConcreteLatticeOperator
spectralData	def	ConcreteLatticeOperator
covarianceConfiguration_covariance	theorem	ConcreteGaussianMoment
latticeCovarianceGJ_gaussian_covariance	theorem	ConcreteGaussianMoment
latticeCovarianceGJ_lift_second_moment	theorem	ConcreteGaussianMoment
latticeEmbedLift	def	LatticeGaussianContinuumConsumer

Declaration (as cited)	Kind	Module
`numberedLatticeCovarianceGJ`	def	`NumberedLatticeGaussianMoment`
`spectralCoefficientEmbedding_inner`	theorem	`LatticeCovarianceRepresentation`
`latticeCovarianceGJ_upstream_coefficients`	theorem	`LatticeCovarianceRepresentation`

TABLE 3. Principal statements and their identifiers. Results are cited by their labels; the right-hand column lists the declaration(s) that carry the content, with a second identifier when the general and the lattice instance are separate declarations.

Result	Identifier(s)
Proposition 3.2	`finiteLaplacian_selfAdjoint`
Proposition 3.3, Corollary 3.4	`finiteLaplacian_neg_semidefinite`
Proposition 3.5	`massOperator_pos_def`
Lemma 3.7	`massOperator_eq_matrix_mulVec`
Corollary 3.8	`massOperatorMatrix_isHermitian`
Theorem 3.9	`massEigenbasis_sum_mul_sum_eq_site_inner`, `massOperator_quadratic_eq_spectral`, `massOperatorMatrix_eigenvalues_pos`
Proposition 4.3	`spectralCovariance_inner`, `latticeCovariance_inner`
Proposition 4.6	`spectralCovarianceGJ_inner`, `latticeCovarianceGJ_inner`
Proposition 5.2	`productGaussian_linear_second_moment_eq`
Proposition 5.3	`productGaussian_linear_second_moment_bound`
Proposition 5.5	`covarianceConfiguration_measurable`
Proposition 5.6	`covarianceConfiguration_memLp`
Theorem 5.7	`covarianceConfiguration_covariance`
Corollary 5.9	`latticeCovarianceGJ_gaussian_covariance`
Proposition 5.11	`finiteCovarianceGaussianMeasure_covariance`
Proposition 5.13	`latticeEmbedLift_eval_eq`, `latticeEmbedLift_continuous`, `latticeEmbedLift_measurable`
Theorem 5.14	`latticeCovarianceGJ_lift_second_moment`, `latticeCovarianceGJ_pushforward_second_moment`
Proposition 5.17	`reindexCovariance_inner`, `numberedLatticeCovarianceGJ_lift_second_moment`
Proposition 6.2	`spectralCoefficientEmbedding_inner`
Proposition 6.3	`spectralCoefficientEmbedding_covariance`
Theorem 6.4	`latticeCovarianceGJ_upstream_coefficients`
Lemma 7.1	`pushforward_eval_L2_norm_le`
Theorem 7.2	`latticeEmbedLift_pushforward_eval_L2_norm_le`, `latticeEmbedLift_uniform_pushforward_eval_L2_norm_le`

APPENDIX B. VERIFICATION LEDGER

Table 4 lists every check that stands behind the claims of this paper, what it establishes, and — the important column — what it does *not* establish. Rows marked "archive" are part of the delivered package and their logs are shipped with it; rows marked "paper" are part of this paper's own audit and are re-run whenever the paper is built.

Table 4: Verification ledger.

Check	Where	Establishes	Does not establish
Build of the whole library	archive	the proof terms typecheck; the reported job count and toolchain	that any statement is the intended one
Axiom report for the public declarations	archive	the footprint of each queried declaration is inside the allowlist; no `sorryAx` on any path	coverage of declarations nobody queries
Injected-axiom control	archive	the firewall fires when an axiom is present	anything about the mathematical content
Interface gate	archive	per-module public declarations are re-queried and their reports are well formed	that modules were chosen to cover everything
Gate parser controls	archive	the gate rejects a custom axiom, `sorryAx`, an empty report, and an error	that its expected-list logic is complete
Upstream body correspondence	archive	the migrated definitions agree textually with the pinned commit	that the mathematics is right, or that the upstream *construction* agrees
Corruption controls	archive	the comparison rejects a dropped inverse power and a changed ℓ^2 index	that no other corruption exists
Hash manifest	archive	the manifest matches the delivered bytes	anything about semantics
Archive hash in the registry	paper	the archive this paper describes is the registered object	that the archive is complete
Comment-stripped `sorry`/`axiom` scan	paper	no `sorry` token and no `axiom` declaration in the sources	that no other escape exists (the axiom report covers this)
Declaration census	paper	the counts printed in this paper	completeness of the syntax-based census
Name gate	paper	every cited Lean name exists in the archive; a fabricated name is rejected	that the statement matches
Metric gate	paper	every printed count was recomputed; a hand-copied count is rejected	that the macro definitions are the right quantities to print
Label and ledger gates	paper	cross-references close; the ledger covers all cited names	that the ledger's kind column is exhaustive
Wording gate	paper	a fixed list of overclaiming phrases is absent	that no other overclaiming occurs

Appendix C. Generated numbers, pins, and drift

Table 5 is the authoritative record of the quantities printed in this paper. Every value in the body is a macro defined in `generated/metrics.tex`, and each is a function of the delivered archive's bytes, computed by `audit/count_lean.py`.

TABLE 5. Derived quantities and their values at the time this paper was built. Every value is a macro defined in `generated/metrics.tex`, which is rewritten from the archive's bytes on every build.

Quantity	Value
core modules (site of the development)	34
Lean modules in the archive, including audit	68
source lines (core modules and root)	3701
declarations of all kinds	211
theorem-level declarations (theorems and lemmas)	149
of which theorems	138
of which lemmas	11
definitions	45
structures	4
instances	5
`sorry` occurrences outside comments	0
`axiom` declarations	0
declarations in the axiom report	64
reporting the allowlisted set	62
reporting no axioms	2
allowlist size	3
interface files gated	27
passing gate checks	41
build jobs	8878
build warnings	42
manifest entries	117
matching the archive bytes	117
files differing from the live tree	2
modules present only in the live tree	3

The pins are: Lean toolchain `leanprover/lean4:v4.34.0-rc2`; Mathlib commit `852d20a84581aeba8527bd01d0be361d8c0e71c0`; upstream `GaussianField` commit `d63a28568a75d99f6cb27af1f888a49a69855a66`. The delivered archive is `20261020_pphi2_clean_finite_lattice.zip` with SHA-256 `39b6ff887baaff0dca9b7edd1a0ab4451f20a256a8e0700a4615638f7c1937db`.

Remark C.1 (Why the paper reports its own drift). Table 5 includes the two drift rows for a reason. A paper that describes a moving source tree is a paper whose numbers cannot be checked after the fact; a paper that describes an archive must say whether the tree still agrees with it. Reporting both keeps the reader from having to guess which snapshot a statement came from, and it makes the (small, declared) divergence a fact rather than a footnote.

REFERENCES

[Dou26] M. R. Douglas, *GaussianField: a Lean 4 development of Gaussian field theory and its lattice covariance*, https://github.com/mrdouglasny/GaussianField, commit `d63a28568a75d99f6cb27af1f888a49a69855a66`, 2026.

[GJ87] J. Glimm and A. Jaffe, *Quantum Physics: A Functional Integral Point of View*, 2nd ed., Springer-Verlag, New York, 1987.

[GRS75] F. Guerra, L. Rosen and B. Simon, *The $P(\varphi)_2$ Euclidean quantum field theory as classical statistical mechanics*, Ann. of Math. **101** (1975), 111–259.

[LU21] L. de Moura and S. Ullrich, *The Lean 4 theorem prover and programming language*, Automated Deduction – CADE 28, Lecture Notes in Comput. Sci. **12699**, Springer, 2021, 625–635.

[mC20] The mathlib Community, *The Lean mathematical library*, Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP 2020), ACM, 2020, 367–381.

[Nel73] E. Nelson, *Probability theory and Euclidean field theory*, Constructive Quantum Field Theory (G. Velo and A. Wightman, eds.), Lecture Notes in Phys. **25**, Springer, Berlin, 1973.

[OS73] K. Osterwalder and R. Schrader, *Axioms for Euclidean Green's functions*, Comm. Math. Phys. **31** (1973), 83–112.

[Sim74] B. Simon, *The $P(\varphi)_2$ Euclidean (Quantum) Field Theory*, Princeton University Press, Princeton, NJ, 1974.